

EMEK

Teknik Mimari ve Geliştirme Blueprint

Web + Mobil + Backend + AI + Optimizasyon + Güvenlik + DevOps

Bu doküman, sohbet boyunca Emek projesi için oluşturulan teknoloji kararlarının, mimarinin, veri modelinin, API sözleşmelerinin, kimlik doğrulama yaklaşımının ve geliştirme planının tek yerde konsolide edilmiş halidir.

Kaynak bağlamı: TÜBİTAK 1812 AGY112 İş Planı + Bölüm-B + teknik tasarım görüşmeleri

0. Yönetici Özeti

Emek, ev içi ve bakım hizmetlerinde müşteri ile bağımsız kadın hizmet sağlayıcıları bir araya getiren iki taraflı bir dijital pazar yeri olarak tasarlanmıştır. Teknik olarak ürünün basit bir ilan/listeme uygulaması olmaması; doğal dil işleme, akıllı eşleştirme, çok kriterli optimizasyon, rota planlama, hizmet oturumu bazlı güvenlik telemetrisi, dijital ispat, şartlı ödeme orkestrasyonu ve ölçülebilir veri analitiği katmanlarını tek bir platformda birleştirmesi hedeflenmektedir.

Temel mimari kararı: mobil uygulama Flutter/Dart, web Next.js/TypeScript, ana backend NestJS/TypeScript, AI/ML/optimizasyon Python, ana veri PostgreSQL + PostGIS, cache/lock Redis, asenkron event altyapısı Google Pub/Sub, compute Cloud Run ve yönetilen Google Cloud bileşenleridir.

En kritik tasarım ilkesi: teknoloji yalnızca “güncel görünsün” diye eklenmemeli; her bileşen somut bir ürün veya Ar-Ge problemini çözmelidir. Bu nedenle Kubernetes, aşırı mikroservisleşme veya her şeyi Python ile yapma yaklaşımı başlangıçta önerilmemektedir. Buna karşılık NLP, matching, optimization ve anomaly detection için Python ayrı servis alanı olarak kullanılmalıdır.

TÜBİTAK açısından güçlü anlatım, “en fazla teknoloji ismini kullanmak” değil; belirli teknolojik belirsizlikleri deneysel olarak çözmek, ölçmek ve sürümler arasında karşılaştırmaktır. Bu nedenle NLP, eşleştirme/optimizasyon, güvenlik/anomali ve gerçek zamanlı düşük gecikmeli platform çalışması temel Ar-Ge eksenleridir.

1. Projenin Teknik Olarak Yeniden Çerçevenmesi

Emek’i yalnızca “Armut benzeri bir hizmet uygulaması” olarak tanımlamak yerine aşağıdaki dört katmanın birleşimi olarak ele almak önerilmektedir:

- Marketplace Platformu: müşteri, sağlayıcı, hizmet, müsaitlik, rezervasyon, ödeme, yorum ve uyumsuzluk yönetimi.
- AI / Decision Engine: doğal dil talebi yapılandırma, aday sağlayıcı çıkarımı, puanlama, çok kriterli eşleştirme ve optimizasyon.
- Safety & Digital Proof: hizmet oturumu bazlı geofencing, telemetri, anomali skoru, panic flow, check-in/out ve Before/After kanıtları.
- Cloud-Native Operations: event-driven mimari, cache, gözlemlenebilirlik, güvenlik, CI/CD ve analitik.

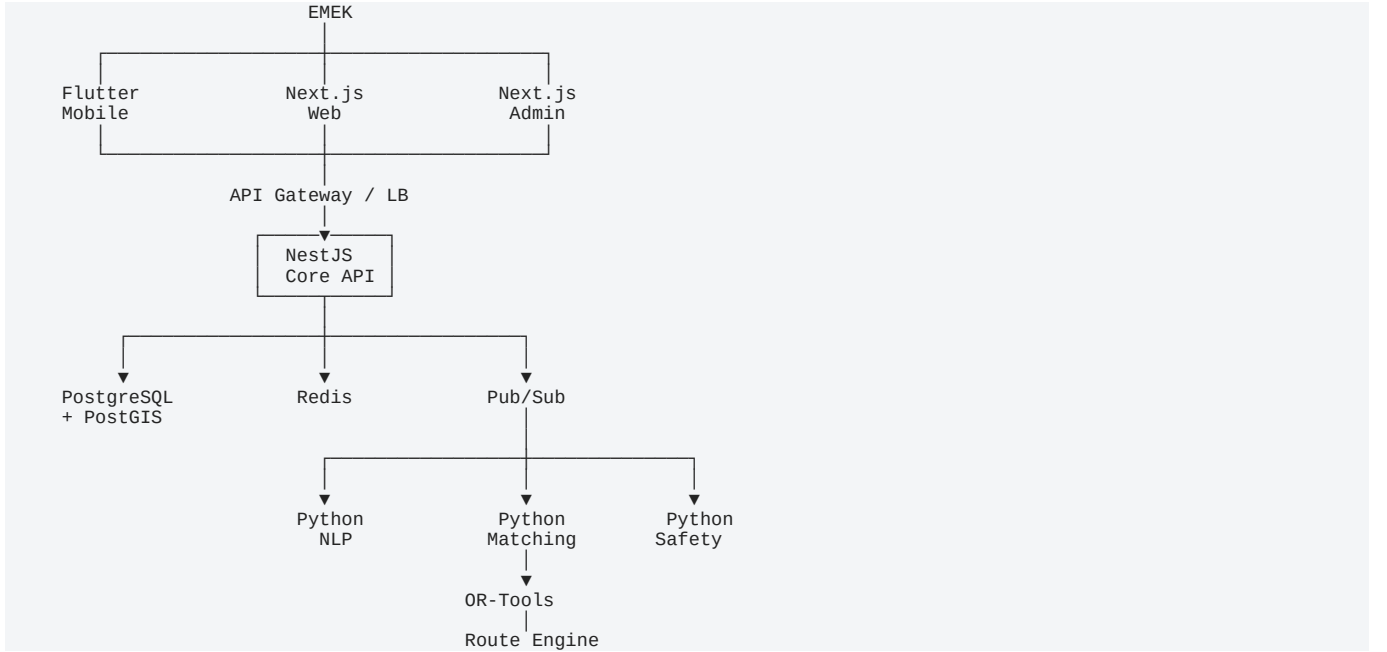
Önerilen üst düzey cümle: “Emek; doğal dil tabanlı hizmet taleplerini yapılandırılmış iş kısıtlarına dönüştüren yapay zekâ katmanı ile coğrafi konum, zaman penceresi, yetkinlik, sağlayıcı kapasitesi, geçmiş performans ve güvenlik değişkenlerini birlikte değerlendiren çok kriterli optimizasyon çekirdeğini; mobil telemetri ve geofencing tabanlı güvenlik mekanizmaları ile olay güdümlü cloud-native altyapı üzerinde birleştiren iki taraflı dijital hizmet pazaryeri platformudur.”

2. Nihai Teknoloji Stack’i

Katman	Teknoloji	Rol
Mobil	Flutter + Dart	iOS/Android tek kod tabanı; gerektiğinde Swift/Kotlin native bridge.
Web	Next.js + TypeScript	Customer web, provider web ve admin/operations.
Core Backend	NestJS + TypeScript	Auth, kullanıcı, provider, service, booking, payment, review, admin, notifications.
AI/ML	Python + FastAPI	NLP, structured extraction, matching, anomaly detection.
Optimization	Python + OR-Tools	Hard/soft constraints, provider assignment, kapasite ve zaman pencereleri.
Ana Veri	PostgreSQL + PostGIS	Transaction-heavy veri ve coğrafi sorgular.
Cache/Lock	Redis / Memorystore	Caching, rate limit, distributed lock, idempotency, sıcak veriler.
Event Bus	Google Pub/Sub	Booking/payment/safety/analytics eventleri.
Compute	Google Cloud Run	Stateless NestJS ve Python servisleri.
Dosya	Google Cloud Storage	Before/After, belgeler ve nesne depolama.

Analytics	BigQuery	Operasyonel, AI ve ESG analitiği.
Auth	Firebase Authentication	Telefon/e-posta OTP ve oturum altyapısı.
Push	Firebase Cloud Messaging	Mobil bildirimler.
Mobile Security	Firebase App Check	Yetkili uygulama istemcilerinden gelen isteklerin doğrulanması.
Realtime	Firestore (opsiyonel)	Chat/presence/temporary realtime state; ana DB değil.
Routing	Google Maps Route Optimization API	Gerçek yol ağı/ETA ve rota servisleri; Emek'in Ar-Ge'sinin yerine geçmez.
Identity	Identity Verification Provider Adapter	EKDS/e-ID/NFC veya yetkili KYC sağlayıcısı gibi seçeneklerin soyutlanması.
Payment	Lisanslı ödeme kuruluşu	Şartlı ödeme, marketplace settlement, refund ve webhook orkestrasyonu.
DevOps	Docker + GitHub Actions + Terraform	Build, test, deploy ve altyapı kodu.
Security	Secret Manager + KMS + IAM + Audit	Secret ve anahtar yönetimi, yetkilendirme ve denetim.
Observability	Cloud Monitoring/Logging + tracing/Sentry	Latency, hata oranı, sağlık ve alarm yönetimi.

3. Genel Sistem Mimarisi



Başlangıç mimarisi “Modular Monolith + bağımsız AI/Optimization servisleri” şeklindedir. Core backend içinde domain modülleri ayrıştırılır; yalnızca bağımsız ölçeklenmesi veya farklı runtime gereksinimi olan AI/optimization/safety parçaları ayrı servis olarak tutulur. Bu yaklaşım küçük ekip için mikroservis yönetim yükünü sınırlarken, ileride parçalı servisleşmeye izin verir.

4. Repository / Klasör Yapısı

```

emek/
├── apps/
│   ├── mobile/      # Flutter
│   ├── web/         # Next.js + TypeScript
│   └── admin/        # Next.js + TypeScript
├── services/
│   ├── api/         # NestJS core backend
│   └── ai/           # Python + FastAPI
├── packages/
│   ├── api-contracts/
│   └── shared-types/

```

```

├── config/
├── infra/
│   ├── terraform/
│   ├── docker/
│   └── github-actions/
├── docs/
│   ├── architecture/
│   ├── api/
│   ├── database/
│   ├── security/
│   └── research/
└── README.md

```

5. NestJS Core Backend Modülleri

```

services/api/src/
├── auth/
├── users/
├── providers/
├── customers/
├── services/
├── skills/
├── availability/
├── bookings/
├── payments/
├── verification/
├── safety/
├── location/
├── reviews/
├── disputes/
├── notifications/
├── admin/
├── audit/
└── common/

```

Örnek booking modülü: controller, service, repository, entity/model, DTO, event producer ve testler ayrı tutulmalıdır. Domain mantığı controller içine gömülmemelidir.

6. Mobil ve Web Uygulamaları

6.1 Flutter Mobile

```

apps/mobile/lib/
├── core/
│   ├── networking/
│   ├── auth/
│   ├── storage/
│   ├── routing/
│   └── analytics/
├── features/
│   ├── onboarding/
│   ├── identity/
│   ├── home/
│   ├── booking/
│   ├── matching/
│   ├── payments/
│   ├── safety/
│   ├── reviews/
│   └── profile/

```

- Provider ve customer deneyimleri aynı uygulama içinde rol tabanlı akışlarla kurgulanabilir.
- Arka plan konum/geofencing gibi OS seviyesinde özelliklerde gerektiğinde Swift/Kotlin native bridge kullanılır.
- GPS takibi 24 saat sürekli olmayacak; aktif hizmet oturumuna bağlanacaktır.

6.2 Next.js Web

```

apps/web/app/
├── (auth)/
├── dashboard/
├── bookings/
├── providers/
├── payments/
├── profile/
└── layout.tsx

apps/admin/app/
├── dashboard/
├── users/
└── verification/

```

- bookings/
- payments/
- safety/
- disputes/
- analytics/
- system/

Admin/Operations web özellikle önemlidir: canlı rezervasyonlar, sağlayıcı doğrulama kuyruğu, güvenlik olayları, ödeme durumları, uyuşmazlıklar, matching analitiği ve sistem sağlığı tek ekranda izlenebilir.

7. PostgreSQL + PostGIS Veri Modeli

Ana işlem verisi PostgreSQL'de tutulur. PostGIS, hizmet alanı, provider konumu, mesafe ve coğrafi filtreleme gibi işlemler için kullanılır. Firestore, varsa chat/presence/temporary realtime state gibi yardımcı alanlarda kullanılabilir; booking/payment/financial ledger ana sisteminin yerine geçmez.

7.1 Ana tablolar

Tablo	Amaç	Kritik not
users	Ana kişi kaydı	1 insan = 1 User
roles / user_roles	Rol yönetimi	Aynı kişi customer + provider olabilir
customer_profiles	Müşteri profili	Tercihler ve müşteri ayarları
provider_profiles	Hizmet sağlayıcı profili	Yetkinlik, kalite ve durum
identity_records	Doğrulanmış kimlik referansı	Provider subject / hash / verification status
verification_attempts	Kimlik doğrulama oturumları	Deneme ve sonuç geçmişi
service_categories / services	Hizmet kataloğu	Temizlik, bakım vb.
skills / provider_skills	Yetkinlik matrisi	Matching girdisi
addresses / provider_service_areas	Konum ve hizmet bölgeleri	PostGIS
availability / availability_exceptions	Takvim	Müsaitlik ve istisnalar
booking_requests	Ham müşteri talebi	raw_text + structured_request
bookings	Gerçek rezervasyon	State machine
booking_status_history	Rezervasyon geçmişi	Auditability
payments / payment_events	Ödeme durumları	Idempotent webhook
safety_sessions / location_events / safety_events	Güvenlik telemetrisi	Hizmet oturumu bazlı
documents	Dosya metadata	Cloud Storage referansları
reviews	İki yönlü değerlendirme	Customer ↔ provider
disputes	Uyuşmazlık yönetimi	Dijital kanıtlarla ilişki
notifications	Bildirimler	Push/in-app
audit_logs	İdari ve kritik işlemler	Değişmez geçmiş için tasarım

7.2 Identity ve multi-accounting tasarımı

```

users
-----
id UUID PK
phone
email
status
created_at
updated_at

user_roles
-----
user_id FK
role

identity_records
-----
id UUID PK
user_id FK
verification_provider
provider_subject_id
identity_hash
verification_level
verification_status
verified_at
created_at

```

Tek kişinin iki ayrı e-posta/telefonla ikinci hesap açmasını engellemek için doğrulanmış kimlik referansı üzerinde veritabanı seviyesinde UNIQUE kısıtı kullanılmalıdır. Bu kısıt yalnızca uygulama koduna değil, doğrudan veritabanına konmalıdır; yarış koşullarında bile mükerrer kaydı son savunma olarak engeller.

Önerilen model: “1 kişi = 1 User”. Aynı User altında Customer Profile ve Provider Profile bulunabilir. Böylece aynı kişinin farklı roller için iki farklı hesabı oluşmaz.

Ham T.C. kimlik numarasını platformun tüm tablolarında dolaştırmak yerine mümkün olduğunda yetkili doğrulama sağlayıcısının subject/reference değeri veya gizli anahtarlı deterministik HMAC tabanlı identity_hash kullanılabilir. Bu, veri minimizasyonuna yardımcı olur; ancak tek başına KVKK yükümlülüklerini ortadan kaldırmaz.

8. Önerilen PostgreSQL DDL (Başlangıç Şeması)

```
-- Extensions
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TYPE user_status AS ENUM ('PENDING', 'ACTIVE', 'SUSPENDED', 'DELETED');
CREATE TYPE app_role AS ENUM ('CUSTOMER', 'PROVIDER', 'ADMIN', 'SUPPORT');
CREATE TYPE verification_status AS ENUM ('PENDING', 'VERIFIED', 'REJECTED', 'EXPIRED');
CREATE TYPE verification_level AS ENUM ('UNVERIFIED', 'PHONE_VERIFIED', 'IDENTITY_VERIFIED', 'PROVIDER_VERIFIED', 'FULLY_VERIFIED');

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  phone VARCHAR(32),
  email VARCHAR(320),
  status user_status NOT NULL DEFAULT 'PENDING',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_login_at TIMESTAMPTZ
);
CREATE UNIQUE INDEX uq_users_email ON users (lower(email)) WHERE email IS NOT NULL;
CREATE UNIQUE INDEX uq_users_phone ON users (phone) WHERE phone IS NOT NULL;

CREATE TABLE user_roles (
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  role app_role NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, role)
);

CREATE TABLE customer_profiles (
  user_id UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  display_name VARCHAR(120),
  preferences JSONB NOT NULL DEFAULT '{}'::jsonb,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE provider_profiles (
  user_id UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  display_name VARCHAR(120),
  bio TEXT,
  experience_years NUMERIC(4,1),
  rating_avg NUMERIC(4,2),
  rating_count INTEGER NOT NULL DEFAULT 0,
  quality_score NUMERIC(6,4),
  verification_status verification_status NOT NULL DEFAULT 'PENDING',
  active BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE identity_records (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL UNIQUE REFERENCES users(id) ON DELETE RESTRICT,
  verification_provider VARCHAR(80) NOT NULL,
  provider_subject_id VARCHAR(255) NOT NULL,
  identity_hash CHAR(64),
  verification_level verification_level NOT NULL DEFAULT 'UNVERIFIED',
  verification_status verification_status NOT NULL DEFAULT 'PENDING',
  verified_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (verification_provider, provider_subject_id),
  UNIQUE (identity_hash)
);

CREATE TABLE verification_attempts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  provider VARCHAR(80) NOT NULL,
  external_session_id VARCHAR(255),
  method VARCHAR(40),
```

```

    status verification_status NOT NULL DEFAULT 'PENDING',
    result_code VARCHAR(80),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    completed_at TIMESTAMPTZ
);

CREATE TABLE service_categories (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(120) NOT NULL,
    description TEXT,
    active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE services (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    category_id UUID NOT NULL REFERENCES service_categories(id),
    name VARCHAR(160) NOT NULL,
    default_duration_minutes INTEGER,
    pricing_model VARCHAR(32) NOT NULL DEFAULT 'FIXED',
    active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE skills (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE provider_skills (
    provider_id UUID NOT NULL REFERENCES provider_profiles(user_id) ON DELETE CASCADE,
    skill_id UUID NOT NULL REFERENCES skills(id) ON DELETE CASCADE,
    verification_status verification_status NOT NULL DEFAULT 'PENDING',
    experience_level VARCHAR(32),
    PRIMARY KEY (provider_id, skill_id)
);

CREATE TABLE addresses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    city VARCHAR(100),
    district VARCHAR(100),
    latitude DOUBLE PRECISION,
    longitude DOUBLE PRECISION,
    location GEOGRAPHY(Point,4326),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_addresses_location ON addresses USING GIST(location);

CREATE TABLE provider_service_areas (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    provider_id UUID NOT NULL REFERENCES provider_profiles(user_id) ON DELETE CASCADE,
    area GEOGRAPHY(Polygon,4326),
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_provider_areas ON provider_service_areas USING GIST(area);

CREATE TABLE availability (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    provider_id UUID NOT NULL REFERENCES provider_profiles(user_id) ON DELETE CASCADE,
    start_time TIMESTAMPTZ NOT NULL,
    end_time TIMESTAMPTZ NOT NULL,
    recurrence_rule TEXT,
    status VARCHAR(24) NOT NULL DEFAULT 'AVAILABLE',
    CHECK (end_time > start_time)
);
CREATE INDEX idx_availability_provider_time ON availability(provider_id, start_time, end_time);

CREATE TABLE availability_exceptions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    provider_id UUID NOT NULL REFERENCES provider_profiles(user_id) ON DELETE CASCADE,
    start_time TIMESTAMPTZ NOT NULL,
    end_time TIMESTAMPTZ NOT NULL,
    reason TEXT
);

CREATE TABLE booking_requests (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id UUID NOT NULL REFERENCES customer_profiles(user_id),
    raw_text TEXT,
    structured_request JSONB,
    parser_version VARCHAR(64),
    parser_confidence NUMERIC(5,4),
    status VARCHAR(32) NOT NULL DEFAULT 'CREATED',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE bookings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    request_id UUID REFERENCES booking_requests(id),
    customer_id UUID NOT NULL REFERENCES customer_profiles(user_id),
    provider_id UUID NOT NULL REFERENCES provider_profiles(user_id),
    service_id UUID NOT NULL REFERENCES services(id),

```

```

    scheduled_start TIMESTAMPTZ NOT NULL,
    scheduled_end TIMESTAMPTZ NOT NULL,
    price_minor BIGINT NOT NULL,
    currency CHAR(3) NOT NULL DEFAULT 'TRY',
    status VARCHAR(40) NOT NULL DEFAULT 'REQUESTED',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (scheduled_end > scheduled_start),
    CHECK (price_minor >= 0)
);
CREATE INDEX idx_bookings_provider_time ON bookings(provider_id, scheduled_start, scheduled_end);
CREATE INDEX idx_bookings_customer_time ON bookings(customer_id, scheduled_start, scheduled_end);

CREATE TABLE booking_status_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
    from_status VARCHAR(40),
    to_status VARCHAR(40) NOT NULL,
    changed_by UUID REFERENCES users(id),
    reason TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE payments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID NOT NULL UNIQUE REFERENCES bookings(id),
    external_payment_id VARCHAR(255) NOT NULL UNIQUE,
    amount_minor BIGINT NOT NULL,
    currency CHAR(3) NOT NULL DEFAULT 'TRY',
    status VARCHAR(40) NOT NULL DEFAULT 'CREATED',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE payment_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    payment_id UUID NOT NULL REFERENCES payments(id) ON DELETE CASCADE,
    external_event_id VARCHAR(255) NOT NULL UNIQUE,
    event_type VARCHAR(80) NOT NULL,
    payload_reference TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE safety_sessions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID NOT NULL UNIQUE REFERENCES bookings(id),
    provider_id UUID NOT NULL REFERENCES provider_profiles(user_id),
    started_at TIMESTAMPTZ,
    ended_at TIMESTAMPTZ,
    status VARCHAR(32) NOT NULL DEFAULT 'INACTIVE'
);

CREATE TABLE location_events (
    id BIGSERIAL PRIMARY KEY,
    safety_session_id UUID NOT NULL REFERENCES safety_sessions(id) ON DELETE CASCADE,
    timestamp TIMESTAMPTZ NOT NULL,
    latitude DOUBLE PRECISION NOT NULL,
    longitude DOUBLE PRECISION NOT NULL,
    accuracy_meters NUMERIC(8,2),
    event_type VARCHAR(40)
);
CREATE INDEX idx_location_events_session_time ON location_events(safety_session_id, timestamp);

CREATE TABLE safety_events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    safety_session_id UUID NOT NULL REFERENCES safety_sessions(id) ON DELETE CASCADE,
    event_type VARCHAR(64) NOT NULL,
    severity VARCHAR(24) NOT NULL,
    risk_score NUMERIC(6,4),
    source VARCHAR(32) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    resolved_at TIMESTAMPTZ
);

CREATE TABLE documents (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID REFERENCES bookings(id) ON DELETE SET NULL,
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    type VARCHAR(40) NOT NULL,
    storage_key TEXT NOT NULL,
    sha256 CHAR(64),
    uploaded_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE reviews (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
    reviewer_id UUID NOT NULL REFERENCES users(id),
    reviewee_id UUID NOT NULL REFERENCES users(id),
    rating NUMERIC(2,1) NOT NULL CHECK (rating >= 1 AND rating <= 5),
    comment TEXT,

```

```

    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    UNIQUE (booking_id, reviewer_id, reviewee_id)
);

CREATE TABLE disputes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    booking_id UUID NOT NULL REFERENCES bookings(id) ON DELETE CASCADE,
    opened_by UUID NOT NULL REFERENCES users(id),
    reason VARCHAR(120) NOT NULL,
    status VARCHAR(32) NOT NULL DEFAULT 'OPEN',
    resolution TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    resolved_at TIMESTAMPTZ
);

CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    type VARCHAR(64) NOT NULL,
    title VARCHAR(180),
    body TEXT,
    data JSONB NOT NULL DEFAULT '{}'::jsonb,
    status VARCHAR(24) NOT NULL DEFAULT 'PENDING',
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE audit_logs (
    id BIGSERIAL PRIMARY KEY,
    actor_user_id UUID REFERENCES users(id),
    action VARCHAR(100) NOT NULL,
    entity_type VARCHAR(80) NOT NULL,
    entity_id UUID,
    old_value JSONB,
    new_value JSONB,
    ip_address INET,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

Bu DDL başlangıç blueprint'idir; production öncesinde migration stratejisi, RLS, retention, index tuning, partitioning ve KVKK veri yaşam döngüsü kurallarıyla birlikte gözden geçirilmelidir.

9. NestJS API Sözleşmesi

API versiyonlaması önerisi: /api/v1. Frontend ve backend arasında rastgele JSON yerine DTO ve ortak contract kullanılması önerilir.

Method	Endpoint	Amaç
POST	/auth/otp/request	OTP gönderimi
POST	/auth/otp/verify	OTP doğrulama ve session
POST	/auth/refresh	Token yenileme
POST	/auth/logout	Oturumu kapatma
GET	/users/me	Kullanıcı profili
PATCH	/users/me	Kullanıcı güncelleme
GET	/users/me/roles	Roller
POST	/verification/session	Kimlik doğrulama session'ı başlatma
GET	/verification/session/:id	Session sonucu
POST	/verification/callback	Provider callback
GET	/verification/status	Kimlik durumu
POST	/providers/profile	Provider profili oluşturma
GET	/providers/me	Provider profili
PATCH	/providers/me	Provider profil güncelleme
GET	/providers/me/skills	Yetkinlikler
POST	/providers/me/skills	Yetkinlik ekleme
GET	/providers/me/earnings	Kazanç özeti
GET	/services	Hizmet kataloğu
GET	/services/:id	Hizmet detayı
GET	/skills	Skill listesi
GET	/providers/me/availability	Müsaitlik
POST	/providers/me/availability	Müsaitlik ekleme
PATCH	/providers/me/availability/:id	Müsaitlik güncelleme
DELETE	/providers/me/availability/:id	Müsaitlik silme
POST	/booking-requests	Talep oluşturma
GET	/booking-requests/:id	Talep detayı
POST	/bookings	Booking oluşturma

GET	/bookings/id	Booking detayı
POST	/bookings/id/confirm	Booking onayı
POST	/bookings/id/cancel	İptal
POST	/bookings/id/check-in	Check-in
POST	/bookings/id/check-out	Check-out
POST	/bookings/id/complete	Tamamlama
POST	/bookings/id/disputes	Uyuşmazlık açma
POST	/bookings/id/reviews	Yorum oluşturma
POST	/safety/sessions/id/panic	Panic flow
POST	/safety/sessions/id/location-events	Konum olayı
GET	/admin/verification-queue	Doğrulama kuyruğu
GET	/admin/safety-events	Güvenlik olayları
GET	/admin/disputes	Uyuşmazlıklar

9.1 Örnek DTO'lar

```
// CreateBookingRequestDto
{
  "rawText": "Cumartesi Kadıköy'de 4 saat detaylı temizlik istiyorum.",
  "addressId": "uuid",
  "preferredStart": "2026-09-26T14:00:00+03:00",
  "preferredEnd": "2026-09-26T19:00:00+03:00"
}

// VerifyIdentitySessionDto
{
  "method": "NFC_EID",
  "provider": "identity-provider-x",
  "successRedirect": "emek://verification/success"
}

// CheckInDto
{
  "bookingId": "uuid",
  "latitude": 40.99,
  "longitude": 29.02,
  "accuracyMeters": 8.3,
  "capturedAt": "2026-09-26T13:58:42+03:00"
}

// PaymentWebhookDto
{
  "eventId": "external-event-id",
  "eventType": "PAYMENT_AUTHORIZED",
  "paymentId": "external-payment-id",
  "timestamp": "2026-09-26T13:57:01+03:00"
}
```

Business error kodları örnekleri: IDENTITY_ALREADY_REGISTERED, PROVIDER_NOT_AVAILABLE, BOOKING_CONFLICT, PAYMENT_FAILED, SAFETY_SESSION_NOT_ACTIVE, DISPUTE_ALREADY_OPEN. Frontend bu kodlara göre kullanıcıya güvenli ve anlaşılır mesaj göstermelidir; ham 500 mesajları kullanıcıya sızdırılmamalıdır.

10. Kimlik Doğrulama, NFC/e-ID, EKDS ve Multi-Account Prevention

NFC/e-ID fikrinin temel mantığı uygundur: kullanıcı uygulamada kimlik doğrulamayı başlatır, uygun cihaz özellikleriyle kart üzerindeki elektronik bilgileri ve/veya yetkili bir kimlik doğrulama hizmetini kullanarak doğrulama yapılır; doğrulama sonucunun bir benzersiz referansı Emek identity kaydına bağlanır.

Ancak NFC'nin tek başına “telefonu tutan kişi = kart sahibi” sonucunu garanti etmediği unutulmamalıdır. Daha yüksek güven seviyesinde identity provider'ın sunduğu ek doğrulamalar (ör. canlılık/yüz eşleştirme) değerlendirilebilir. Biyometrik veriler devreye girdiğinde KVKK açısından çok daha sıkı kurallar oluşur.

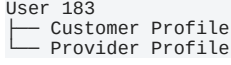
EKDS/KDHS erişimi veya resmi veri kaynaklarına doğrudan bağlanma varsayımı yerine bir Identity Verification Adapter Layer kurulması önerilir. Böylece yetkili bir servis, e-ID/NFC çözümü veya ticari KYC sağlayıcısı değiştiğinde core backend değişmez. Erişilebilen yöntem ve izinler başvuru ve uygulama öncesinde ilgili kurum/sağlayıcılarla doğrulanmalıdır.

10.1 Önerilen kimlik akışı

```
REGISTER
↓
Phone OTP
```



10.2 Tek kullanıcı / çok rol modeli



Bu yapı, aynı kişinin müşteri ve sağlayıcı olarak platformda bulunmasına izin verirken ikinci doğrulanmış kişi kaydı oluşturmalarını engeller.

10.3 Account Recovery

Multi-accounting önleminin karşılığı mutlaka bir hesap kurtarma akışı olmalıdır. Eski telefon numarasına erişimini kaybeden doğrulanmış kullanıcı, yeni iletişim bilgisini ek doğrulama ile bağlayabilmelidir. Aynı identity referansı bulunduğu yeni hesap açmak yerine mevcut User'a güvenli şekilde dönülür.

10.4 Verification Levels

```

UNVERIFIED
PHONE_VERIFIED
IDENTITY_VERIFIED
PROVIDER_VERIFIED
FULLY_VERIFIED
  
```

Müşteride identity verified yeterli olabilirken sağlayıcı için identity + belge/yetkinlik + safety onboarding gibi ek koşullar uygulanabilir. “Doğrulandı” rozeti ile “adli sicili temizdir” gibi daha geniş ve hassas iddialar birbirinden ayrılmalıdır.

11. Uygulamanın Baştan Sona Çalışma Akışı

Adım	Sistem davranışı
1. Kayıt	Telefon/e-posta OTP + kullanıcı profili + gerekli identity onboarding.
2. Kimlik	Identity provider üzerinden doğrulama; tekil User identity kaydı.
3. Hizmet talebi	Müşteri serbest metin yazabilir veya yapılandırılmış form kullanabilir.
4. NLP	Python metni hizmet türü, tarih, zaman penceresi, süre, konum ve ihtiyaçlara ayırır.
5. Candidate Retrieval	PostgreSQL/PostGIS + availability + skill + verification filtreleri.
6. Matching	Python scoring/ranking; hard constraints elme, soft constraints puanlama.
7. Optimization	OR-Tools ile daha geniş assignment / kapasite / zaman / seyahat hedefleri.
8. Route	Gerekirse Google Maps Route Optimization / routing ile gerçek ETA/mesafe.
9. Öneri	Müşteriye uygun sağlayıcı, fiyat, ETA ve temel eşleşme gerekçesi gösterilir.
10. Ödeme	Lisanslı PSP ile payment authorization/hold.
11. Provider kabulü	Provider booking'i kabul eder; state makinesi ilerler.
12. Hizmet günü	Geofence + check-in + safety session.
13. Hizmet	Telemetry + safety rules + anomaly score; panic button acil akışı.
14. Kanıt	Before/After dosyaları Cloud Storage; metadata database'de.
15. Check-out	Hizmet oturumu kapanır.
16. Tamamlama	Müşteri onayı; gerekiyorsa dispute window.
17. Settlement	Ödeme sağlayıcısının settlement/payout akışı.
18. Review	İki taraflı değerlendirme + kalite metrikleri.

12. Booking State Machine

```

REQUESTED
↓
MATCHED
↓
PROVIDER_PENDING
↓
CONFIRMED
↓
PAYMENT_AUTHORIZED
↓
SCHEDULED
↓
PROVIDER_ARRIVING
↓
CHECKED_IN
↓
IN_PROGRESS
↓
CHECKED_OUT
↓
CUSTOMER_CONFIRMED
↓
COMPLETED
↓
SETTLED

```

Yan durumlar: CANCELLED / DISPUTED / SAFETY_HOLD

State geçişleri sadece service/controller koduyla değil; izin verilen geçiş tablosu ve booking_status_history ile denetlenmelidir. Payment ve safety gibi kritik yan akışlar state geçişleriyle birlikte idempotent ve audit edilebilir tasarlanmalıdır.

13. AI / NLP / Matching / Optimization Motoru

13.1 NLP

Kullanıcı örneği: “Yarın öğleden sonra Kadıköy’de kedili ev için 4 saat detaylı temizlik istiyorum.” Python servisi bunu makine işleyebilir bir yapıya dönüştürür.

```

{
  "service_type": "deep_cleaning",
  "duration_minutes": 240,
  "district": "Kadikoy",
  "date": "2026-09-20",
  "time_window": ["13:00", "18:00"],
  "requirements": ["pet_friendly"]
}

```

LLM/NLP katmanı “kullanıcı ne istedi?” sorusuna odaklanır. “Kime atayalım?” kararı ayrı candidate retrieval + constraint + optimization katmanına bırakılır. Bu ayrım güvenilirlik, açıklanabilirlik ve test edilebilirlik açısından önemlidir.

13.2 Candidate Retrieval

```

Örnek filtreleme:
10,000 provider
  ↓ service type
2,100
  ↓ availability
630
  ↓ skills
190
  ↓ verification
87
  ↓ geographic constraints
~80 candidates

```

13.3 Hard vs Soft Constraints

Hard constraint	Soft constraint
Doğrulama zorunlu	Mesafe
Müsaitlik zorunlu	Rating

Gerekli skill zorunlu	Experience
Rezervasyon çıkışması yok	Customer preference
Mesafe/time threshold	Travel time / cost
Kapasite sınırı	Continuity / previous interaction

13.4 Objective / Scoring

ÖRNEK (başlangıç ağırlıkları örnektir):

```
Score =
  0.25 × skill_match
+ 0.20 × availability
+ 0.15 × quality
+ 0.15 × distance
+ 0.10 × rating
+ 0.15 × preference
```

Ar-Ge: ağırlıkları ve objective fonksiyonunu deneysel olarak optimize edin.

13.5 OR-Tools Optimization

OR-Tools provider assignment, kapasite, zaman penceresi ve rota benzeri kısıtların optimizasyonunda kullanılabilir. Google Maps Route Optimization API, gerçek yol ağı ve ETA gibi dış altyapı ihtiyacını karşılayabilir; ancak bu servis Emek'in Ar-Ge karar motorunun kendisi değildir. Emek'in özgünlüğü provider + customer + skills + availability + quality + safety + geography değişkenlerini birlikte değerlendiren objective/constraint yapısında olmalıdır.

13.6 Explainability

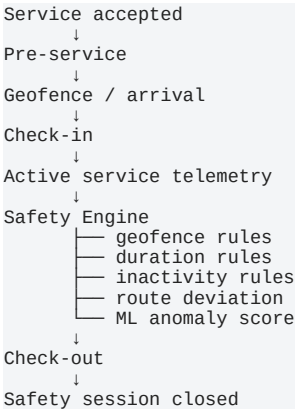
Neden bu provider?

- ✓ Hizmet yetkinliği
- ✓ Uygun takvim
- ✓ 2.7 km mesafe
- ✓ Doğrulanmış profil
- ✓ Müşteri tercihlerine uyum

Matching sonucuyla birlikte kullanılan skor bileşenlerinin ve model sürümünün saklanması; hem kullanıcı açıklaması hem TÜBİTAK deneyleri hem de hata ayıklama için değerlidir.

14. Safety / Telemetry / Anomaly Detection

Güvenlik sistemi 24 saat sürekli GPS takibi yerine hizmet oturumuna bağlanmalıdır: hizmet kabul edildi → provider yolda → geofence → check-in → aktif hizmet → check-out → tracking sona erer. Bu yaklaşım teknik gereksinim, batarya tüketimi ve veri minimizasyonu arasında daha dengeli bir model sunar.



Safety kararı tamamen ML'e bırakılmamalıdır. Panic button gibi kritik eylemler deterministic rule-based flow ile anında işlenmelidir. ML anomali skoru destekleyici sinyal olur.

14.1 Örnek Safety Signals

- GPS trajectory / route deviation
- Geofence enter/exit
- Expected duration vs actual duration
- Check-in / check-out time

- Device motion / inactivity
- Panic button
- ML anomaly score

14.2 Risk seviyesi

NORMAL
WARNING
HIGH_RISK
EMERGENCY

15. Ödeme, Şartlı Ödeme ve Dijital İspat

Emek kendi escrow/emanet hesabını kurmamalıdır. Lisanslı ödeme kuruluşunun marketplace/alt satıcı/ödeme yönlendirme yetenekleri üzerinden şartlı ödeme, settlement, refund ve webhook orkestrasyonu yapılmalıdır. Emek kendi veritabanında payment intent/reference/status/event bilgilerini tutar.

CREATED
↓
AUTHORIZED
↓
HELD
↓
SERVICE_COMPLETED
↓
RELEASE_PENDING
↓
RELEASED

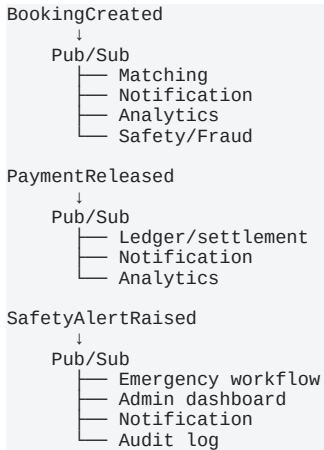
Alternatif: FAILED / REFUNDED / DISPUTED

Ödeme webhook'ları idempotent olmalıdır. Aynı event iki kez geldiğinde settlement iki kere yapılmamalıdır; payment_events.external_event_id üzerinde UNIQUE kısıtı bu korumaya yardımcı olur.

15.1 Digital proof

Check-in/out timestamp, geofence olayı, Before/After fotoğrafları, ödeme eventleri ve dispute kaydı bir booking evidence graph olarak ilişkilendirilebilir. Fotoğraflar Cloud Storage'da; metadata, hash, booking_id ve timestamp Postgres'te tutulabilir.

16. Event-Driven Architecture / Pub/Sub



Pub/Sub sayesinde core backend her alt sistemi senkron biçimde beklemek zorunda kalmaz. Event isimleri ortak sözleşmeyle yönetilmelidir.

Önerilen event sözlüğü: UserRegistered, IdentityVerified, ProviderApproved, BookingCreated, BookingMatched, ProviderAccepted, BookingConfirmed, PaymentAuthorized, PaymentReleased, PaymentRefunded, ServiceStarted, ServiceCompleted, SafetyAlertRaised, DisputeOpened, ReviewCreated.

17. Admin / Operations Platform

EMEK OPERATIONS

Active Bookings 126
 Active Services 38
 Verified Providers 2314
 Pending Verification 24
 Open Disputes 7
 Active Safety Alerts 2
 Today's GMV XXXX TL

Canlı işlemler / Safety alerts / Matching queue / Verification queue / Payment status

Admin sistemi yalnızca CRUD paneli olmamalıdır; operasyonel karar desteği vermelidir. Matching score dağılımı, safety alerts, payment state, provider availability ve verification queue gibi teknik metrikler izlenebilmelidir.

18. Güvenlik ve KVKK Tasarımı

Güvenlik katmanı “authentication + authorization + data protection + auditability + runtime controls” şeklinde ele alınmalıdır.

Alan	Yaklaşım	Amaç
Authentication	Firebase Auth / secure session	OTP, session lifecycle
Authorization	RBAC	Customer / Provider / Admin / Support
Client integrity	Firebase App Check	İstemci uygulaması doğrulama
Transport	HTTPS/TLS	Tüm ağ trafiği
Secrets	Secret Manager	API key, password, certificate vb.
Keys	Cloud KMS	Kriptografik anahtar yönetimi
Database	Postgres access controls	En az yetki
Object storage	Signed/private access	Fotoğraf ve belgeler
Audit	audit_logs	Kritik işlemler
Rate limiting	Redis/API gateway	Brute force ve abuse azaltma
Testing	SAST/dependency scan/pentest	CI ve periyodik güvenlik testleri

T.C. kimlik numarası, kimlik doğrulama çıktıları, adli sicil gibi alanlar veri minimizasyonu ile ele alınmalıdır. Özellikle ceza mahkûmiyeti/güvenlik tedbirleriyle ilişkili veriler KVKK kapsamında özel nitelikli veri olarak değerlendirilir. Biyometrik doğrulama kullanılırsa biyometrik veriler için de özel nitelikli veri kuralları devreye girer. Bu nedenle ham belgeyi veya gereksiz kişisel veriyi sınırsız süreyle saklamak yerine doğrulama sonucu, provider reference ve gerekli audit bilgileri saklanmalıdır.

Bu bölüm teknik tasarımıdır; gerçek veri işleme envanteri, hukuki dayanak, açık rıza/aydınlatma gereklilikleri, saklama-imha politikaları ve kurum/sağlayıcı sözleşmeleri hukuk uzmanı tarafından doğrulanmalıdır.

19. Cloud / Infra / DevOps

Google Cloud

- Cloud Run
 - NestJS Core API
 - Python AI Service
- Cloud SQL
 - PostgreSQL + PostGIS
- Memorystore
 - Redis
- Pub/Sub
- Cloud Storage
- BigQuery
- Secret Manager
- Cloud KMS
- Cloud Monitoring / Logging

Başlangıçta Kubernetes kullanılmaması önerilir. Cloud Run + Pub/Sub + Cloud SQL + Redis, ekip kapasitesine göre daha düşük operasyonel yükte ölçeklenebilir bir cloud-native temel sağlar. Kubernetes gelecekte gerçek ölçek/operasyon gereksinimi oluştuğunda değerlendirme konusu olabilir; teknoloji göstergesi olarak eklenmemelidir.

19.1 CI/CD

```

git push
↓
Pull Request
↓
Lint
↓
Unit Tests
↓
Integration Tests
↓
Security Scan
↓
Docker Build
↓
Artifact Registry
↓
Deploy to Staging
↓
Smoke Tests
↓
Production

```

19.2 Environments

```

development
staging
production

```

Staging ortamı pilot ve TÜBİTAK demo testleri için production'a yakın bir kopya olmalıdır. Terraform ile altyapı tekrar üretilebilir hale getirilmelidir.

20. Observability / Monitoring

Ölçülmesi gereken teknik platform metrikleri: API p50/p95 latency, RPS, error rate, database connection health, queue/event lag, Cloud Run instance scaling, cache hit rate, safety event latency ve mobile crash-free rate.

```

Platform KPI
├── API p50 latency
├── API p95 latency
├── Requests/sec
├── Error rate
├── Event processing lag
├── DB health
├── Cache hit rate
└── Mobile crash-free sessions

```

21. Test Stratejisi

Test türü	Kapsam	Ne zaman
Unit	Domain, DTO validation, scoring functions	Her PR
Integration	Postgres, Redis, Pub/Sub, payment adapters	Her PR / staging
E2E	Customer → booking → payment → service → review	Staging
Load	100/250/500 eşzamanlı booking gibi senaryolar	Release candidate
Security	Dependency scan, secret scan, auth abuse, pentest	Periyodik
AI evaluation	NLP F1, matching Recall@K, anomaly recall/FPR	Model release
Mobile	iOS/Android GPS, geofence, background behavior	Cihaz matrisi
Chaos / failure	Duplicate webhook, provider timeout, Pub/Sub retry	Staging

22. TÜBİTAK Açısından Ar-Ge ve Ölçülebilirlik

Teknoloji değerlendirmesinde güçlü anlatım, “modern teknoloji kullandık” değil, “teknolojik belirsizliği şu deneylerle çözdük” yaklaşımıdır. Bu nedenle dört ana deney eksenı önerilir.

Alan	Ar-Ge sorusu	Metrikler
NLP	Doğal dili yapısal talebe dönüştürme	Precision / Recall / F1 / slot extraction F1
Matching	Doğru provider adaylarını ve sıralamasını bulma	Recall@K / Acceptance Rate / latency p50-p95
Optimization	Çok kriterli assignment ve seyahat hedefleri	Travel time reduction / distance reduction / constraint violation / utilization
Safety	Normal ve anomaliyi ayırma	Recall / false positive rate / detection latency
Platform	Gerçek zamanlı servis performansı	p50/p95 latency / RPS / error rate / availability

22.1 Model sürümlerinin kaydı

```

booking_requests
raw_text
structured_request
parser_version
parser_confidence

booking_match_results (öneri)
booking_id
provider_id
overall_score
skill_score
distance_score
availability_score
quality_score
preference_score
algorithm_version
selected
created_at

```

Bu kayıtlar sayesinde v1/v2/v3 model veya algoritma sürümleri karşılaştırılabilir. A/B ve offline evaluation sonuçları araştırma dokümantasyonuna aktarılmalıdır.

22.2 Örnek deney çerçevesi

Baseline	Proposed
manual/simple filter	NLP + constraints + optimization

Compare:

- matching acceptance
- Recall@10
- p95 latency
- travel time
- distance
- constraint violations
- safety false positives

23. Geliştirme Yol Haritası

Faz	Ana çıktılar
Faz 1 — Foundation	GitHub, Docker, NestJS, Next.js, Flutter, PostgreSQL, Auth, RBAC, temel CI/CD
Faz 2 — Marketplace Core	Provider, service, skills, availability, booking, calendar
Faz 3 — Identity	Identity session, provider adapter, unique identity, recovery, provider verification
Faz 4 — AI	NLP, structured request, candidate retrieval, matching, explainability
Faz 5 — Optimization	OR-Tools, hard/soft constraints, route integration, benchmark
Faz 6 — Payment	Payment provider, intent, webhook, settlement, refund, dispute
Faz 7 — Safety	Geofence, check-in/out, telemetry, panic, anomaly detection
Faz 8 — Admin/Ops	Verification, booking, payment, safety, dispute and AI dashboards
Faz 9 — Analytics	Pub/Sub, BigQuery, operational/AI/ESG metrics

24. Ne Yapılmalı / Ne Yapılmamalı

Karar	Öneri
Yapılmalı	Her teknoloji bir iş problemi veya Ar-Ge hipoteziyle ilişkilendirilmeli.
Yapılmalı	Core backend NestJS/TypeScript; AI/optimization Python olarak

	ayrılmalı.
Yapılmalı	PostgreSQL + PostGIS ana veri katmanı olmalı.
Yapılmalı	Redis cache/lock/idempotency için kullanılmalı.
Yapılmalı	Pub/Sub ile event-driven alt akışlar kurulmalı.
Yapılmalı	Identity adapter ile sağlayıcı bağımsız kimlik mimarisi kurulmalı.
Yapılmalı	1 kişi = 1 User; Customer/Provider profile ayrımı kullanılmalı.
Yapılmalı	NLP → candidate retrieval → constraints → optimization zinciri kurulmalı.
Yapılmalı	Safety'de rules + ML hibriti kullanılmalı.
Yapılmalı	Booking/payment state machine ve audit trail kurulmalı.
Yapılmamalı	Her şeyi Python ile yazmak.
Yapılmamalı	Firestore'u transaction-heavy ana finansal database yapmak.
Yapılmamalı	İlk sürümde 20-30 mikroservis kurmak.
Yapılmamalı	Kubernetes'i sırf modern görünmek için eklemek.
Yapılmamalı	Kendi escrow/ödeme altyapısını kurmak.
Yapılmamalı	LLM'e kritik eşleştirme ve safety kararlarını kontrolsüz bırakmak.
Yapılmamalı	Kullanıcıların 24 saat sürekli GPS'ini izlemek.
Yapılmamalı	AI için yalnız "%85 doğruluk" gibi tek başına açıklamasız metrikler kullanmak.
Yapılmamalı	E-Kimlik/EKDS erişiminin doğrudan mevcut ve garanti olduğu varsayımını dokümana kesin gerçek gibi koymak.
Yapılmamalı	Adli sicil veya kimlik belgelerini gereksiz süre ve kapsamda saklamak.

25. Mimari Kararların Gerekçeleri

Neden NestJS? Web ve backend tarafında TypeScript ortaklığı, modüler controller/service yapısı, validation, event/queue entegrasyonu ve ekip geliştirme hızını destekler. Next.js ile aynı dil ailesi kullanılır.

Neden Python sadece gerektiği yerde? NLP, ML, OR-Tools ve veri bilimi ekosistemi nedeniyle AI/optimization kısmında Python doğal avantaj sağlar. Core CRUD/transaction/business logic'i Python'a taşımak zorunlu değildir.

Neden PostgreSQL? Booking, payment, role, availability ve state transition gibi ilişkisel, transaction-sensitive veriler için güçlü tutarlılık gerekir. PostGIS coğrafi sorguları sağlar.

Neden Cloud Run? Stateless container servisleri için yönetilen, otomatik ölçeklenen ve küçük ekipte operasyon yükünü azaltan bir compute katmanıdır.

Neden Pub/Sub? Booking/payment/safety eventlerini gevşek bağlı biçimde dağıtarak core request path'ini hafifletir ve gelecekte servislerin ayrı ölçeklenmesini sağlar.

Neden Kubernetes değil? Başlangıçta operasyonel karmaşıklığı artırır. Gerçek yük ve organizasyon gereksinimi oluştuğunda değerlendirilebilir.

Neden Firestore sadece yardımcı? Realtime/chat gibi alanlar için hızlı olabilir; ancak finansal ve karmaşık ilişkisel çekirdekte PostgreSQL daha uygun bir ana sistemdir.

26. TÜBİTAK / Demo Günü İçin Teknik Hikâye

Teknik demo sırasında en etkili senaryo; tek bir booking'ın baştan sona izlenmesi ve karar motorunun neden/ nasıl çalıştığının gösterilmesidir.

1. Customer writes request
2. NLP extracts structured request
3. Map shows candidate providers
4. Hard constraints remove invalid candidates
5. Matching ranks candidates
6. Optimization selects assignment
7. Explainability panel shows reasons
8. Payment authorization created
9. Provider accepts
10. Geofence + check-in activates safety session
11. Telemetry/anomaly engine runs
12. Check-out + evidence closes service
13. Payment settlement occurs

14. Review updates quality metrics
15. Admin analytics receives events

Admin panelinde ayrıca “Matching version”, “Risk score”, “Payment state” ve “Audit trail” alanlarının gösterilmesi teknik derinliği görünür hale getirir.

27. Mevcut Proje Dokümanında Teknoloji Açısından Revize Edilmesi Önerilen Noktalar

Mevcut ifade yaklaşımı	Teknik olarak daha güvenli ifade
“Firebase Cloud Functions + Firestore mikroservisleri”	“Cloud Run tabanlı stateless servisler + Pub/Sub + PostgreSQL/PostGIS; Firestore gerekiyorsa realtime yardımcı katman”
“AI ile eşleştirme”	“LLM/NLP structured extraction + candidate retrieval + hard/soft constraints + çok kriterli optimization”
“Rota optimizasyonu”	“Emek objective/constraint engine + OR-Tools; gerçek yol ağı/ETA için Route Optimization API”
“GPS tracking”	“Hizmet oturumu bazlı telemetri + geofence + rules + anomaly scoring”
“Escrow”	“Lisanslı ödeme kuruluşu üzerinden şartlı ödeme / settlement orchestration”
“%85 NLP accuracy”	“Precision/Recall/F1 + slot extraction F1 + confidence ve test veri seti”
“Sektörde ilk”	Kanıtlanmadıkça “literatür/patent/rekabet taraması sonucunda ayrıışan” gibi doğrulanabilir dil
“Sıfır maliyetli serverless”	“Tüketim bazlı/otomatik ölçeklenebilir cloud-native altyapı”

28. Ürün, Operasyon ve İş Modeli ile Teknolojinin Kesiştiği Noktalar

- C2C iki taraflı yapı nedeniyle matching ve trust sistemi ürünün merkezi olmalı.
- Kadın sağlayıcıların sermayesiz onboarding'i için peşin teklif ücreti yerine platform içi işlem ve kalite ekonomisi esas alınabilir; teknik sistem ödeme ve komisyon akışını desteklemelidir.
- Provider tarafında takvim, kazanç, verification ve safety ekranları customer tarafı kadar önemlidir.
- B2B/ESG tarafı için operasyonel olaylardan ölçülebilir kadın istihdamı ve hizmet metrikleri üretmek üzere BigQuery/analytics katmanı kurulabilir.
- Platform dışına kaçışı azaltmak için ürün faydaları, garanti/SLA ve güvenlik mekanizmaları kullanılabilir; ancak manipülatif veya kullanıcıyı cezalandırmaya dayalı lock-in tasarımları hukuki ve etik açıdan ayrıca değerlendirilmelidir.

29. Kimlik Doğrulama İçin Örnek Teknik Sözleşmeler

29.1 Start session

POST /api/v1/verification/session

Request

```
{
  "method": "NFC_EID",
  "provider": "identity-provider-x",
  "purpose": "ACCOUNT_VERIFICATION"
}
```

Response

```
{
  "sessionId": "ver_123",
  "status": "PENDING",
  "clientToken": "short-lived-token"
}
```

29.2 Verification result

```
{
  "sessionId": "ver_123",
  "status": "VERIFIED",
  "verificationLevel": "IDENTITY_VERIFIED",
  "providerSubjectId": "external-subject",
  "verifiedAt": "2026-09-20T10:00:00+03:00"
}
```

Ham kimlik verilerinin örnekte gösterilmemesi kasıtlıdır; production sözleşmesi mümkün olan en az veri ilkesiyle tasarlanmalıdır.

Teknik çalışma dokümanı — mimari öneriler, veri modeli, API sözleşmeleri ve geliştirme planı

30. İlk Kodlama Sırası — Gün Gün Değil, Bağımlılık Sırasına Göre

1. Monorepo oluştur: apps/mobile, apps/web, apps/admin, services/api, services/ai, packages/*, infra/.
2. Docker + local development orchestration + environment management.
3. PostgreSQL + PostGIS migration altyapısı ve ilk tablolar.
4. NestJS core modules: users, roles, services, provider, availability.
5. Firebase Auth + backend token verification + RBAC.
6. Flutter onboarding/login/profile ve Next.js customer/provider temel ekranları.
7. Booking request + booking state machine + calendar.
8. Identity adapter + mock provider + unique identity ve recovery.
9. Python FastAPI + NLP structured extraction + evaluation harness.
10. Candidate retrieval + matching + explanation.
11. OR-Tools optimization + benchmark harness.
12. Payment provider sandbox + webhook idempotency + payment state machine.
13. Safety session + geofence + panic flow + telemetry.
14. Admin/Ops dashboard ve audit logs.
15. Pub/Sub eventler + BigQuery analytics.
16. CI/CD + staging + load/security testing + production readiness.

31. İlk Sürümde Bilerek Sonraya Bırakılabilecekler

- Kubernetes ve service mesh
- 30 ayrı mikroservis
- İleri seviye gerçek zamanlı ML pipeline'ları
- Kendi ödeme altyapısı
- Geniş çaplı global routing optimizer
- Çok karmaşık recommendation/personalization
- Gereksiz blockchain/crypto benzeri teknoloji eklemeleri

Bunların sonraya bırakılması teknik olarak “basit proje” anlamına gelmez. Aksine çekirdek problemi önce sağlam kurup ölçek ihtiyacı olduğunda teknoloji eklemek daha sürdürülebilir bir mimari stratejidir.

32. Final Technical Checklist

Alan	Son karar
Architecture	Modular monolith + AI/optimization services
Mobile	Flutter + Dart + 必要 native bridges
Web	Next.js + TypeScript
Backend	NestJS + TypeScript
AI	Python + FastAPI
DB	PostgreSQL + PostGIS
Cache	Redis
Events	Pub/Sub
Cloud	Cloud Run + Cloud SQL + Storage + BigQuery
Identity	Provider adapter + NFC/e-ID where supported + unique identity
Payment	Licensed provider orchestration
Safety	Geofence + telemetry + rules + anomaly ML
Optimization	OR-Tools + routing provider
DevOps	Docker + GitHub Actions + Terraform
Security	RBAC + App Check + Secret Manager + KMS + audit
Quality	Unit + integration + E2E + load + security + AI evaluation
Research	Versioned datasets/models + measurable benchmarks

33. Resmî / Teknik Kaynaklar ve Doğrulama Notları

Aşağıdaki kaynaklar mimaride anılan teknoloji ve mevzuat başlıklarının başlangıç referanslarıdır. Sürüm, fiyat, API erişimi, lisanslama veya program çağırısı gibi değişebilir bilgiler uygulama öncesinde güncel resmî kaynaktan tekrar doğrulanmalıdır.

TÜBİTAK 1812 BiGG Yatırım — <https://tubitak.gov.tr/tr/destekler/sanayi/ulusal-destek-programlari/1812-yatirim-tabanli-girisimcilik-destek-programi-bigg-yatirim>

Next.js Documentation — <https://nextjs.org/docs>

NestJS Documentation — <https://docs.nestjs.com/>

Flutter Documentation / Supported Platforms — <https://docs.flutter.dev/reference/supported-platforms>

Google Cloud Run — <https://cloud.google.com/run/docs>

Google Cloud Pub/Sub — <https://cloud.google.com/pubsub/docs/overview>

Cloud SQL for PostgreSQL — <https://cloud.google.com/sql/docs/postgres>

PostGIS — <https://postgis.net/>

Google OR-Tools — <https://developers.google.com/optimization/>

Google Maps Route Optimization — <https://developers.google.com/maps/documentation/route-optimization/overview>

Firebase App Check — <https://firebase.google.com/docs/app-check>

Firebase Authentication — <https://firebase.google.com/docs/auth>

KVKK — T.C. Kimlik Numaralarının İşlenmesi Hakkında Rehber — <https://www.kvkk.gov.tr/Icerik/7798/Turkiye-Cumhuriyeti-Kimlik-Numaralarinin-Islenmesi-Hakkinda-Rehber>

KVKK — Özel Nitelikli Kişisel Veriler — <https://www.kvkk.gov.tr/Icerik/8184/Ozel-Nitelikli-Kisisel-Verilerin-Islenmesine-Iliskin-Rehber>

Resmî Gazete — EKDS yönetmeliği bağlamı — <https://www.resmigazete.gov.tr/eskiler/2021/10/20211015-3.htm>

34. Sohbet Boyunca Verilen Proje Kararlarının Konsolide Özeti

Bu bölüm, konuşma boyunca verilen teknik kararları tek bir kontrol listesinde toplar. Amaç önceki değerlendirmelerde geçen hiçbir ana teknik önerinin kaybolmamasıdır.

- Emek'in temel teknik konumu: basit bir hizmet ilanı değil; marketplace + AI decision engine + safety + digital proof + payment orchestration platformudur.
- Core backend Python'a taşınmak zorunda değildir; NestJS/TypeScript ana backend olarak önerilmiştir.
- Python, özellikle NLP, ML, matching, OR-Tools optimization ve anomaly detection için ayrı servis alanıdır.
- Web Next.js/TypeScript; mobil Flutter/Dart; gerektiğinde Swift/Kotlin native bridge.
- Ana database PostgreSQL + PostGIS; Firestore yalnızca realtime/chat/presence gibi yardımcı alanlarda.
- Redis cache, rate limit, distributed lock, idempotency ve hot availability data için.
- Pub/Sub booking/payment/safety/notification/analytics eventleri için.
- Cloud Run backend ve Python servislerinin compute katmanı; Cloud SQL Postgres, Cloud Storage, BigQuery, Secret Manager, KMS ve Monitoring destekleyicidir.
- İlk sürümde Kubernetes, service mesh ve aşırı mikroservisleşme önerilmemektedir.
- Kimlik mimarisinde 1 kişi = 1 User; Customer Profile ve Provider Profile aynı kullanıcıya bağlanabilir.
- Kimlik doğrulama için NFC/e-ID fikri uygundur; ancak NFC tek başına kart sahibi eşitliğini kanıtlamaz, provider/liveness gibi ek doğrulamalar değerlendirilebilir.
- EKDS veya belirli kimlik sağlayıcısına doğrudan bağımlılık yerine Identity Verification Adapter kurulmalıdır.
- Multi-accounting için identity reference üzerinde DB UNIQUE constraint kullanılmalıdır; recovery flow şarttır.
- Ham TCKN mümkün olduğunca yaygın tutulmamalı; provider subject veya güvenli HMAC hash gibi referans tasarımı düşünülmelidir.
- Adli sicil ve biyometrik veriler daha sıkı KVKK yaklaşımı gerektirdiğinden yalnız gerekli doğrulama sonucu/metadata saklanmalıdır.
- Booking state machine kullanılmalı; ödeme ve safety eventleri audit edilebilir ve idempotent olmalıdır.

- AI akışı: raw text → NLP → structured request → candidate retrieval → hard constraints → scoring → optimization → explanation.
- LLM bütün kararları tek başına vermemeli; kritik matching/safety kuralı ve ölçülebilir bir karar motoruyla kontrol edilmelidir.
- Google Maps routing, Emek'in özgün Ar-Ge'sinin yerine geçmemeli; sadece yol/ETA altyapısı olarak kullanılmalıdır.
- Güvenlik sistemi hizmet oturumuna bağlı olmalı; 24 saat sürekli konum takibi yapılmamalıdır.
- Panic flow ML beklememeli; deterministic emergency workflow ile anında işlenmelidir.
- Before/After dosyaları Cloud Storage; metadata ve booking bağlantısı Postgres.
- Ödeme kendi escrow sisteminizle değil lisanslı ödeme sağlayıcısı üzerinden koşullu ödeme/settlement orkestrasyonu ile tasarlanmalıdır.
- Admin dashboard operasyonel zekâ katmanı gibi tasarlanmalı; verification, safety, disputes, matching ve payment durumlarını göstermelidir.
- TÜBİTAK sunumunda teknoloji sayısından çok teknik belirsizliklerin deneysel çözümünü ve benchmarkları görünür kılın.
- NLP için Precision/Recall/F1, matching için Recall@K ve latency, optimization için travel-time/distance/constraint metrics, safety için recall/FPR/detection latency, platform için p50/p95/RPS/error rate ölçülmelidir.
- Versiyonlama: NLP parser version, matching algorithm version ve safety model version tutulmalı; model/algoritma karşılaştırmaları kayıt altına alınmalıdır.
- Geliştirme sırası: foundation → marketplace core → identity → AI → optimization → payment → safety → admin → analytics.
- Kodlamaya Flutter ekranlarıyla dağınık şekilde başlamak yerine domain model → database → API contract → backend → AI → web/mobile şeklinde ilerlemek daha güvenlidir.

35. Sonuç: Emek İçin Tek Cümlelik Teknik Yol Haritası

Emek için hedeflenen yapı; Flutter mobil istemci ve Next.js web istemcilerinin, NestJS tabanlı transaction-oriented core backend ile konuştuğu; PostgreSQL/PostGIS ve Redis'in ana veri katmanını oluşturduğu; Pub/Sub ile event-driven çalışan; Python tabanlı NLP, matching, OR-Tools optimization ve safety/anomaly servislerinin karar motorunu oluşturduğu; kimlik doğrulama ve ödeme sağlayıcılarının adapter katmanlarıyla entegre edildiği; Cloud Run üzerinde Dockerized olarak çalışan; Terraform + GitHub Actions ile tekrarlanabilir şekilde dağıtılan; güvenlik, audit ve analytics katmanlarıyla izlenebilen bir cloud-native marketplace mimarisidir.

Bu mimari “çok teknoloji = yüksek teknoloji” yaklaşımı yerine “doğru problem = doğru teknoloji = ölçülebilir çıktı” yaklaşımına dayanır. Projenin teknolojik iddiası özellikle NLP tabanlı talep yapılandırma, çok kriterli eşleştirme/optimizasyon, hizmet oturumu bazlı güvenlik/anomali tespiti ve bu karar motorlarının gerçek zamanlı bir iki taraflı pazaryerinde güvenilir biçimde çalıştırılmasıdır.

Uygulama öncesi zorunlu doğrulama konuları: identity/EKDS/KDHS erişim modeli, ödeme sağlayıcısının marketplace/conditional payout yetenekleri, KVKK veri envanteri ve saklama politikası, GPS/safety veri retention, Apple/Google background location koşulları, üçüncü taraf API kullanım şartları ve güncel TÜBİTAK çağrı koşulları.